IEEE Open Journal of
Circuits and Systems

Received 1 July 2024; revised 5 September 2024 and 9 October 2024; accepted 11 November 2024. Date of current version 25 June 2025.

Digital Object Identifier 10.1109/OJCAS.2024.3502641

BAG3++: An Extensible Generator Framework for Automated Layout-Aware AMS Design

FELICIA GUO (Graduate Student Member, IEEE), BOB ZHOU (Graduate Student Member, IEEE), AYAN BISWAS (Member, IEEE), PAUL KWON, ZHAOKAI LIU (Member, IEEE), KEN HO (Graduate Student Member, IEEE), VLADIMIR STOJANOVIĆ (Fellow, IEEE), AND BORIVOJE NIKOLIĆ (Fellow, IEEE)

Electrical Engineering and Computer Sciences, University of California at Berkeley, Berkeley, CA 94720, USA
This article was recommended by Associate Editor M. Saligane.

CORRESPONDING AUTHORS: F. GUO AND B. ZHOU (e-mail: felicia_guo@berkeley.edu; bob.linchuan@berkeley.edu)

This work was supported in part by the National Science Foundation (NSF) AI Institute for Advances in Optimization under Award 2112533; in part by Google; and in part by Intel SRS.

ABSTRACT We present BAG3++, an extensible analog/mixed-signal (AMS) design framework for layout-aware design. BAG3++ realizes a unified design environment that merges schematic, layout, and verification views into a single development interface. We further introduce new automated design features that enable rapid automation and optimization across a range of performance specifications, processes, and applications. We demonstrate the practical use of these features through (a) a bit-reconfigurable successive-approximation-register (SAR) analog-to-digital converter (ADC) implemented in the open-source Skywater 130nm process and (b) an ultra-high speed output driver optimized in two modern processes. BAG3++ interfaces with both commercial and open-source design frameworks, and the extensibility of BAG3++ is further illustrated through the integration of an open-source simulator.

INDEX TERMS Analog mixed-signal (AMS) circuits, electronic design automation (EDA), high-speed serial link, integrated circuit layout, open-source hardware, SAR ADC.

I. INTRODUCTION

CONVENTIONAL analog and mixed-signal (AMS) design has traditionally relied on iterative design loops spanning from schematic design to layout to extraction, with multiple hand-offs between design, verification, and layout engineers at each subsequent design stage. However, the delineation between schematic design and subsequent layout-dependent steps is increasingly challenged by (a) layout-driven effects in modern processes, (b) inconsistent scaling trends in wire and transistor parasitics and (c) complex layout design rules. Collectively, such considerations have expanded both design iteration time and total required design cycles.

Incorporation of layout-aware design is therefore critical to controlling development overheads, with layout automation being the primary means targeted by multiple prior works [1], [2], [3]. In ALIGN [1], ML/heuristic algorithms are used to perform subgraph isomorphism comparison operations to identify design primitives that guide the automated creation of performance and layout constraints. However, this procedure limits layouts to predefined blocks, and may not be easily extensible to highly performant custom designs. FASoC [2] enables analog layout automation through the use of automated place-and-route (PnR) tools for connectivity, thus allowing for rapid startup times in new processes. However, this approach encounters difficulties in the high frequency regime, which (a) often lies outside the typical characterization range of digital libraries and (b) nullifies simplifying parasitic assumptions employed in routing heuristics. Moreover, use of digital routing tools restricts analog cell layouts to standard-cell-adherent form factors, and makes the framework more suited for primarily digital designs.

The Berkeley Analog Generator (BAG), first presented in [4], proposed a generator-driven circuit design methodology that codified templates for producing unique circuit instances through parameterized input specifications. Critically, this approach provided designers with a

© 2024 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

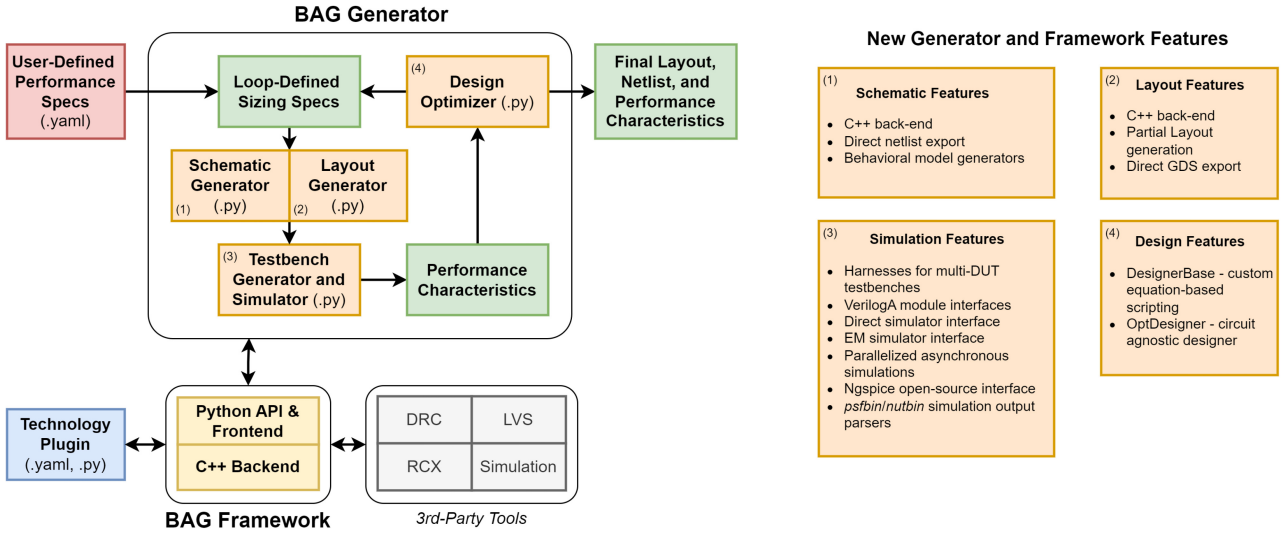


FIGURE 1. BAG3++ flow structure and new features. In addition to defining desired specifications (red), users must also implement Python BAG3++ circuit generators (orange) through our front-facing framework API (yellow). Using the framework, the generator can iterate and realize the target circuit instance (green).

disciplined means of codifying design methods for schematic, layout, and verification generators, thus significantly accelerating the design process *at all stages*. BAG2 [3] improved on the original work by enabling process portability through the introduction of abstract primitives classes. The framework was employed to produce fabricated designs in more than eight different process technologies by both academic and industrial users [5], [6]. However, inefficiencies in backend implementation and incompatibilities with open-source tooling presented challenges.

This paper presents BAG3++, a silicon-proven, extensible analog design framework compatible with both commercial and open-source tooling. Like BAG2, BAG3++ starts with a generator application programming interface (API) that enables schematic parametrization and layout automation. A new backend now cleanly decouples BAG3++ from commercial library and tooling dependencies while also maintaining competitive design compilation times and quality of results for designed circuits. BAG3++ also introduces new simulation APIs, most notably *OptDesigner*, that accelerate highly parallelized design characterization, exploration, and sizing. We illustrate the practical use of these enhancements through two design examples - a successive-approximation-register (SAR) analog-digital converter (ADC) design generator tested in the open-source Skywater 130nm process and a ultra-high speed wireline output driver capable of transmitting at 130 Gbps in a 45 nm SOI process and the Intel 16nm process.

II. BAG3++: CORE DESIGN INFRASTRUCTURE FEATURES

The core structure and flow of BAG3++ are shown in Fig. 1. BAG's core framework provides a Python-based frontend API for writing highly parameterizable circuit generators. Schematic generators codify the circuit architecture and parametric sizing, while layout generators codify detailed placement and routing strategies. These generators produce circuit instances based on user-defined sizing parameters and technology-specific rules. BAG framework also has APIs for writing testbench generators for simulating and characterizing circuits. Finally, to wrap these all together, BAG provides APIs for design optimization, allowing the designer to consume the simulation results and iterate on the circuit sizing toward the desired performance specifications. All these features are built upon BAG's internal interfaces to 3rd-party tools for running simulation and physical verification (LVS, DRC, RCX).

To realize a unified design environment that seamlessly transitions between schematic, layout, and verification views, BAG3++ features a rearchitected C++ backend that provides speed-up relative to the previous Python-based implementation. This implementation has enabled sophisticated new design routines that have helped augment and transform the analog design flow used in BAG3++. The following sections detail these features.

A. GENERATOR APIs ENABLING AGILE DESIGN

At the schematic level, BAG3++ introduces a new intermediate representation (IR) that represents circuit as structured hierarchy trees. This IR is leveraged by a new behavioral model generation API to create behavioral mirrors for designs, thus allowing for early system design exploration and specification. This IR is further employed to emit schematic and layout netlists of various formats (SystemVerilog, CDL, Spectre, SPICE), thus allowing for flexible interfacing with external tools.

At the layout level, the monolithic layout generation step found in BAG2 and other established analog frameworks has been replaced by a flexible partial layout scheme. This mechanism allows designers to select only specific components and routes to be laid out, thus providing realistic extracted performance through accurate capture of parasitics

at critical layout junctures (e.g., inductors and routing to transmitters). Importantly, the partial layout schemes flexibly sidesteps the overhead of complete layout generation for parasitic estimates. A practical example of rapid iteration between schematic and layout steps using this mechanism is provided in Section IV. In similar theme to improvements made in the schematic generation API, BAG3++ natively emits GDS-format layouts through backend C++ routines that compute device placements at the polygon level. Finally, to enable interoperability with other analog design frameworks and flows, BAG3++ supports the import of non-BAG-generated GDS layouts.

B. FLEXIBLE SIMULATION AND POST-PROCESSING

For simulation testbenches, BAG3++ directly interfaces with simulators through the native netlist generation facility and implements a simulation backend that avoids heavy SKILL-based APIs associated with commercial test suites. These features provide ergonomic benefits by (a) cutting simulator interfacing runtimes and (b) allowing users to wrap standard simulation facilities in higher-level Python code (e.g., writing a Python function that creates for-loops calling transient simulations on each pass). The last point is heavily leveraged to pair the circuit generators with verification suites that combine testbench reuse, e.g., for a class of ADC architectures, with highly customized code, e.g., specific analysis and reporting for a given design target. Examples of these suites are discussed in Sections III and IV.

Open-source simulator integration was previously limited by the use of proprietary interfacing formats associated with parsing the results produced by commercial simulators. However, BAG3++ is able to directly parse two new output simulation formats: *psfbin* and *nutbin*. The *psfbin* format is the native output of the Cadence commercial simulator Spectre and is supported in BAG3++ through an openly available package [7]. The *nutbin* format is part of the open SPICE3 specification and is supported by open-source simulators, e.g., Ngspice. These backend features allow BAG3++ to work with state-of-the-art commercial tools while also maintaining seamless support for open-source tools. Users can transition between modes with minimal modification to existing setups, as discussed in Section V.

C. AUTOMATED DESIGN SUITE

The generator and simulation backends provide for codifying sophisticated circuit design procedures in BAG3++. The versatility of the API and the rich variation of applicable design approaches (equation-based methods, iterative loops, etc.) are shown in the later design example sections. In particular, a major new feature is the *OptDesigner* design tool that performs automatic transistor sizing in a *completely circuit-agnostic manner*. *OptDesigner* ingests as inputs a circuit generator, evaluation metrics, a testbench, and the desired design parameters (e.g., transistor widths) with feasible sweep limits. With these collaterals, *OptDesigner* generates a multi-dimensional design space from its parameters, with initial points populated by a parametric sweep. With the SciPy package, *OptDesigner* performs an interpolation between initial points to produce a continuous performance model for the given design generator [8]. Subsequently, multivariate optimization functions including minimization methods such as *BFGS* can be applied to discover optimal design points within the continuous model. Critically, the discovered point may exist outside the original sweep granularity provided by the designer. This provides two benefits: (a) enabling faster coarse-grain parametric experiments, and (b) organic discovery of new optimal points. A complete design flow illustrating specification, generation, and optimization steps is shown in Fig. 2.

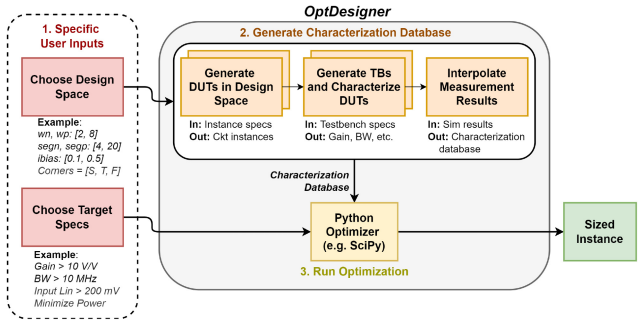


FIGURE 2. ***OptDesigner*** **flow, with sample specifications for a 5-T OTA. Note that generation and characterization of each DUT is independent and can be highly parallelized.**

III. DESIGN INSTANCE: OPEN-SOURCE SAR ADC

This section details the use of BAG3++ to design a sophisticated circuit system, the SAR ADC, in the open-source Skywater 130nm process. Importantly, the generator is capable of realizing reproducible silicon-verified design instances with 4 to 8-bits of resolution. This section specifically highlights BAG3++'s ability to do simultaneous high-level system planning and low-level layout-aware subblock design. The presented ADC generator is open-source and is based on an open-source PDK. However, the simulations were performed with commercial tools due to the need for accurate large-signal noise characterizations.

From a design perspective, resolution, sampling bandwidth, input range, and throughput are the critical performance metrics for an ADC. However, translating these directly into low-level transistor implementation is a difficult optimization problem. We thus leverage BAG3++'s API to plan sub-block design specifications (Fig. 3). This process ingests the targeted metrics, technology parameters, and application information to first derive block-level constraints and specifications such as LSB size and a total noise constraint. These derived parameters are used as feasible initial constraints for sub-blocks, which are optimized in their own unique generators (explained below). Through a tiered approach, the generator is able to propagate high-level system specifications to sub-blocks, thus breaking apart

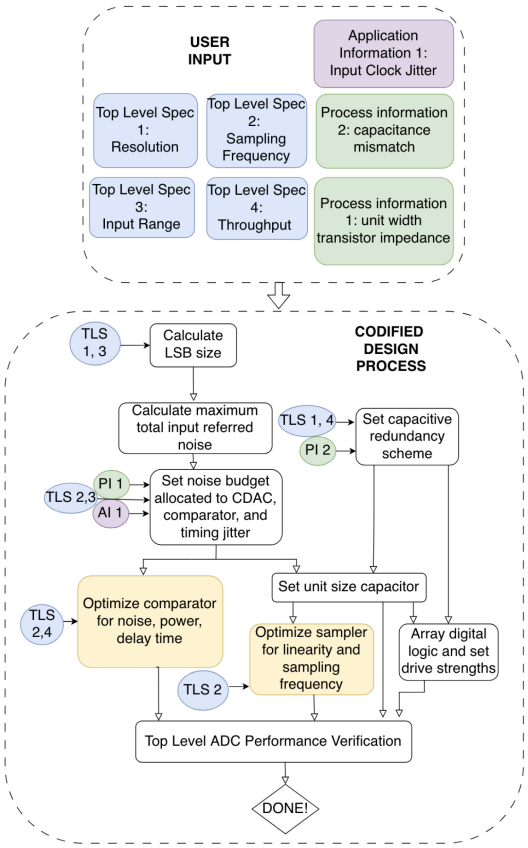


FIGURE 3. Codified design flow for ADC. Blue indicates top level specifications, purple indicates application information, green indicates process information. Optimization blocks done with *OptDesigner* are highlighted in yellow.

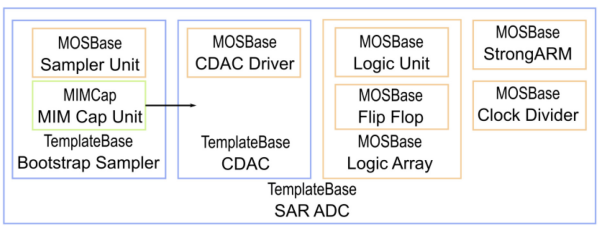


FIGURE 4. High-level ADC generator structure.

a complex design problem with an intuitive hierarchical solution.

Complementing the hierarchical design flow is a synchronous SAR ADC layout generator that is implemented with BAG3++'s layout API (Fig. 4). The main circuit blocks are a charge-sharing capacitive DAC, comparator, digital control block, and counter-based clock divider (Fig. 5).

A. TRANSISTOR SIZING

The sampler and comparator blocks are designed with the *OptDesigner* feature. Topologically, the sampler generator supports (a) a NMOS sampler, (b) a bootstrapped sampler [9], and (c) omitting the sampler. The last option is used for testing or in cases where an end-user would like to implement a custom circuit.

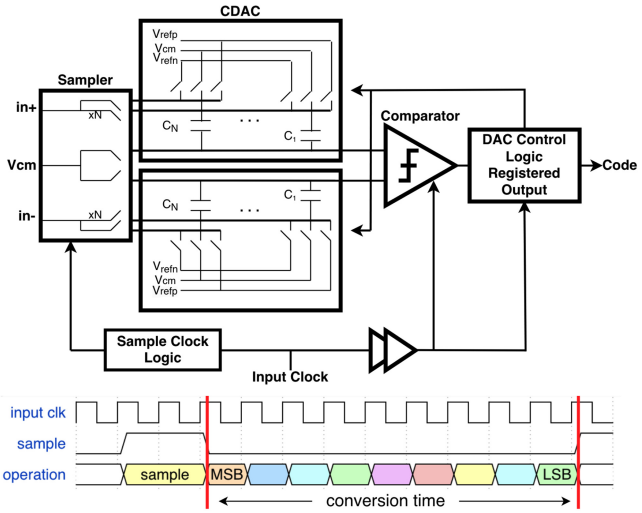


FIGURE 5. Synchronous ADC block diagram with timing diagram. Converted code output at the end of conversion time (second red bar).

By using *OptDesigner*, we design the sampler for maximum linearity given a sampling frequency constraint. The comparator (StrongARM) was designed for low noise given regeneration time. This is achieved by selecting critical transistors for sizing (e.g., input pair, current tail for the comparator) and identifying key metrics to create a design space on which *OptDesigner* can construct a continuous performance model. The design flow derives our constraints from user inputs (Fig. 3). An example that optimizes the comparator design with respect to average power consumption, subject to noise and delay constraints, is shown in Fig. 6.

B. CAPACITOR DAC

Code redundancy is a means of improving the post-silicon robustness of an ADC, as it allows for code errors to be digitally re-calibrated. It also improves the sampling speed of the ADC by allowing for greater tolerance of DAC settling errors and reducing the size of the largest capacitor in the array. This capacitor reduction in turn minimizes the settling time of the conversion step for the MSB, which is beneficial for our synchronous design as the input clock period can be further reduced.

User-defined throughput and resolution specifications are used to calculate the redundancy (in LSBs) at the schematic level for each decision step k defined by the following equation [10]:

$$q(i) = -p(i+1) + 1 + \sum_{k=i+2}^{M} p(k) \tag{1}$$

where $p(i)$ is the weight for the i-th decision step and M is the total number of conversion steps. The 0th decision step corresponds to the most significant bit of the ADC output code.

Assuming the mismatch of unit capacitors follows a Gaussian distribution with mean of C_u and standard deviation

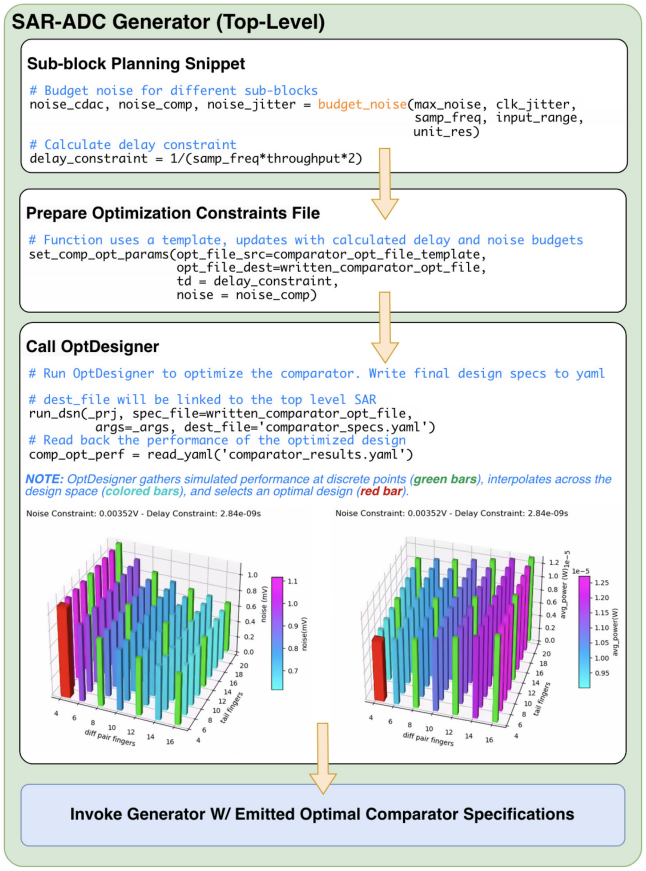


FIGURE 6. Example sub-block design flow with *OptDesigner* with visualization across two dimensions of design space.

$\sigma_u C_u$, where σ_u is the standard deviation per mean the mean of the redundancy of the i-th bit can be written as:

$$\mu_i = q(i)C_u \tag{2}$$

where C_u is the unit capacitance.

The standard deviation of the redundancy of the i-th bit is:

$$\sigma_i = \sigma_u C_u \sqrt{p^2(i+1) + 1 + \sum_{k=i+2}^{M} p^2(k)} \tag{3}$$

It is desirable to size the number of redundant bits for each decision step such that the mean redundancy is greater than the standard deviation of the redundancy. Formally, we can define a ratio:

$$r_i = \frac{\mu_i}{\sigma_i} = \frac{q(i)C_u}{\sigma_u C_u \sqrt{p^2(i+1) + 1 + \sum_{k=i+2}^{M} p^2(k)}} \tag{4}$$

As the unit capacitance, C_u, cancels out and the standard deviation per mean σ_u is defined by the process, we use the following quantity for optimization, which only depends on the CDAC weighting:

$$r_i' = \frac{q(i)}{\sqrt{p^2(i+1) + 1 + \sum_{k=i+2}^{M} p^2(k)}} \tag{5}$$

To optimize for all array weights, the following equation is used [11]:

$$\max \min_{i \le M-2} r_i' \tag{6}$$

The generator implemented can only generate integer numbers of unit-sized capacitors that can be arranged into set numbers of columns in the capacitor array layout. Therefore, limits are imposed on the weights that may be used. Though BAG3++ does not natively solve mixed non-linear programming problems, the versatility of its Python-based interface allows us to integrate any available Python package. In this case, the capacitor weight problem is solved using APOPT [12].

The implemented generator produces bottom-plate-sampled capacitor charge-sharing DACs, laid out in unit-sized metal-insulator-metal (MIM) capacitors. As capacitance mismatch from layout organization can degrade the performance of the final ADC, the generator further includes the capability to add dummy columns for each capacitor. Finally, large capacitors can be laid-out in two separate blocks arranged symmetrically about the vertical midpoint of the capacitor array, producing better layout matching. For this feature, the end user only needs to specify the smallest DAC index that should be split, with the generator conducting correct automatic placement and routing.

C. SAMPLING AND DIGITAL LOGIC

To adapt to the changing requirements of throughput and resolution, the sampling signal and number of digital logic bits must also be configurable. The key feature enabling this automatic adjustment is BAG3++'s parameterized sub-block generation. The counter-based sample-signal generator can be configured for different fixed ratios to accommodate a total ADC latency of up to 16 input clock cycles; 2 input clock cycles are used for sampling, while the rest are used for converting the sampled input. Similarly, DAC control logic may be reconfigured to the appropriate number of DAC decision steps. Output registers are buffered to sufficient drive strength, and output codes can be read from these registers in parallel.

D. EXTRACTED SIMULATIONS

The presented generator has been evaluated with ADC instances ranging from 4-bits to 8-bits of resolution [13].

The generated instances were simulated with extracted post-layout parasitics. All static simulations showed under 1 bit of INL and DNL (Fig. 8). All dynamic simulations were taken by using a 4.25 MHz 3.2 V differential full-scale input signal and showed approximately 50 dBFS of dynamic range. Achievable maximum sampling frequency, average power, and layout area are summarized in Table 1, and can be referenced as limitations of the design flow in Skywater 130nm.

The layout generation time for a single ADC is about 15 seconds, and the schematic generation time is about 60 seconds.

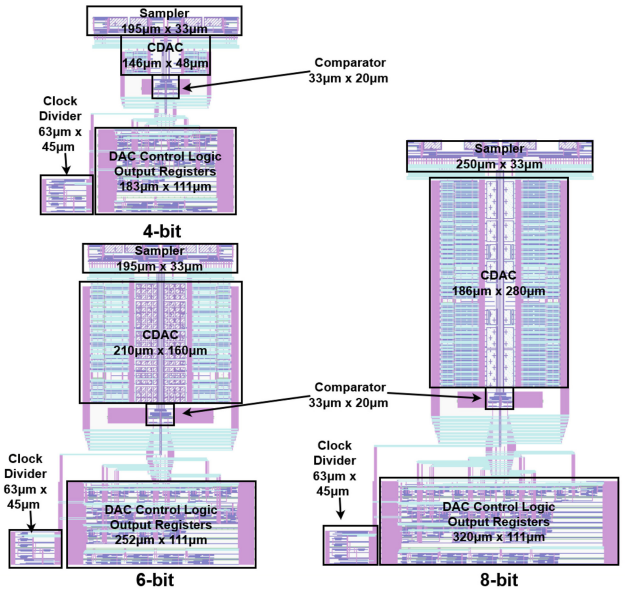


FIGURE 7. Physical layout of 4-bit, 6-bit, and 8-bit configurations in Skywater 130nm, to scale.

TABLE 1. ADC generator dynamic results summary. These are specifications that may be achieved by the design flow.

Configuration	Max. Sampling Frequency	Avg. Power at Max. Freq.	Area
4 bit 4-cycle conversion	52 MHz	2.02 mW	264 μm x 276 μm
6 bit 7-cycle conversion	35 MHz	2.43 mW	327 μm x 431 μm
8 bit 9-cycle conversion	21 MHz	1.67 mW	395 μm x 570 μm

E. SILICON MEASUREMENTS

The 8-bit design of the ADC was fabricated in the Skywater 130nm process. The INL and DNL remained under one LSB (Fig. 10). Post-integration results yielded a slight performance degradation in clock bandwidth due to the length of the clock routing (Fig. 10), as well as measured dynamic range (Fig. 11). Average power was measured to be 2.2 mW on silicon. The increased power may be in part due to the addition of test board decoupling capacitors. As BAG3++ has been available in the open-source, this layout has already been recreated by members of the open-source community.

IV. DESIGN INSTANCE: ULTRA-HIGH SPEED WIRELINE OUTPUT DRIVER

This section presents the design of an ultra-high speed wireline output driver using *OptDesigner* [14], and further demonstrates BAG3++'s ability to be extended to highly performant and custom designs. The design flow is shown in Fig. 12.

The schematic of the device-under-test (DUT) and test-bench (TB) is illustrated in Fig. 13. The driver is an NMOS differential pair with a 50 Ω resistor load and shunt-peaking inductors. The resistors isolate the parasitic capacitance

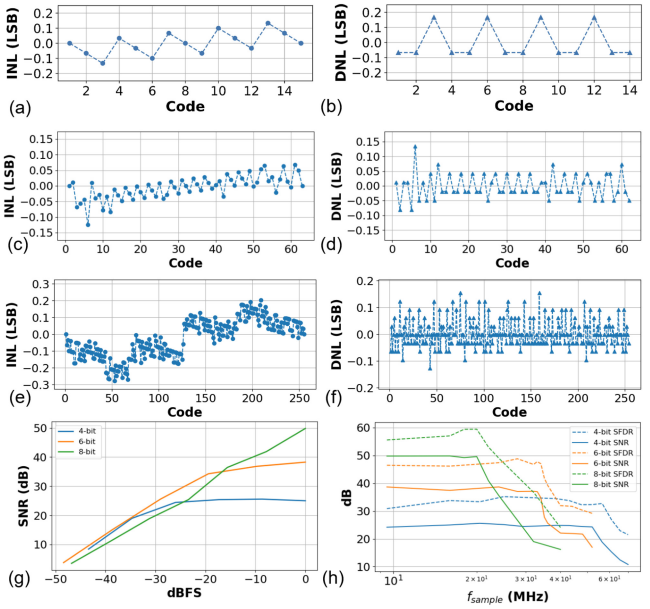


FIGURE 8. (a) INL of 4-bit ADC (b) DNL of 4-bit ADC (c) INL of 6-bit ADC (d) DNL of 6-bit ADC (e) INL of 8-bit ADC (f) DNL of 8-bit ADC (g) Dynamic range plots (h) SNR and SFDR plots.

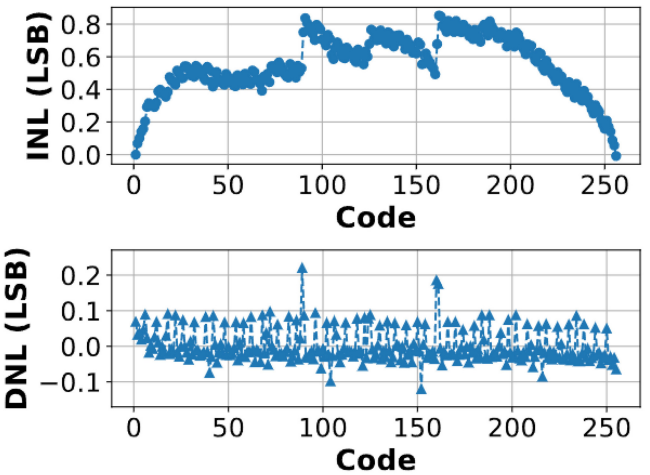


FIGURE 9. Integrated ADC measurements. Static linearity results for INL and DNL. Measured with 3.2 V differential amplitude, 500 KHz signal, 11.18 MHz sample frequency.

of the shunt peaking inductors from the high frequency output node, thereby removing them from the total output capacitance that has to be equalized by shunt peaking. The load is represented by the 80 fF capacitors which are a simplified lumped model of electrostatic discharge (ESD) and channel traces for the purpose of this demonstration. In a real design scenario, the reduced model can be replaced by the real channel s-parameter model for more accurate simulation results.

A. SUB-BLOCK DESIGN WITH EM SIMULATION AND PARTIAL LAYOUT GENERATION

The use of inductors to increase the bandwidth in this driver design naturally necessitates optimal inductor design. Typically, a designer would create inductor layouts and

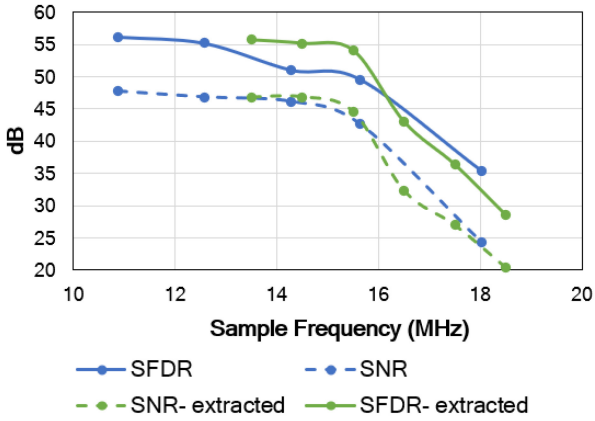


FIGURE 10. Comparison of post-silicon and extracted integrated ADC measurements. Measured with 3.2 V differential amplitude, 4.25 MHz signal.

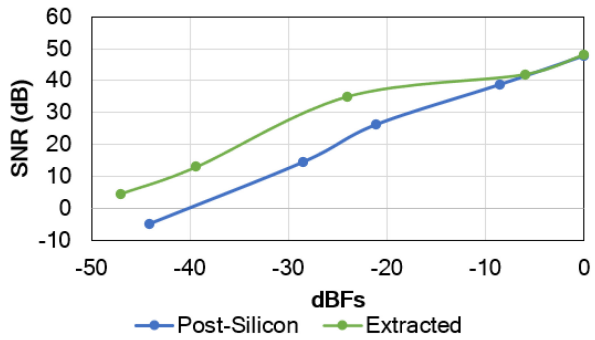


FIGURE 11. Comparison of post-silicon and extracted integrated ADC measurements. Measured with 4.25 MHz differential signal, full scale amplitude 3.2 V. Dynamic range on silicon was about 40 dBFs.

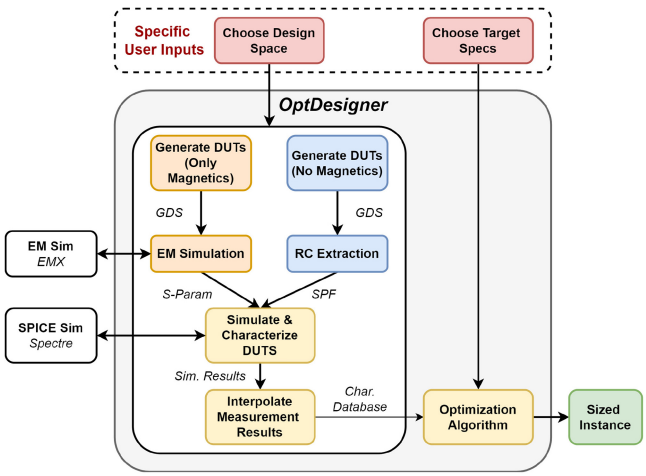


FIGURE 12. Design flow for the ultra-high speed wireline output driver using *OptDesigner*.

extract S-parameters using an electromagnetic (EM) simulator. However, this extraction must be done carefully to reflect the parasitic effects of routing and placement. This is especially critical since routing magnetics play a significant role at ultra-high bandwidths.

BAG3++'s partial layout feature assists with this design problem by enabling generation of only the inductor and

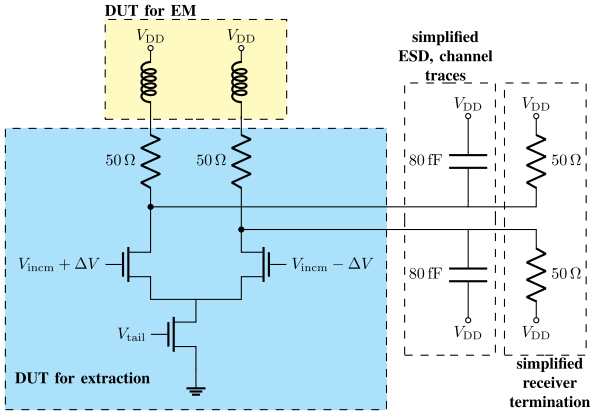
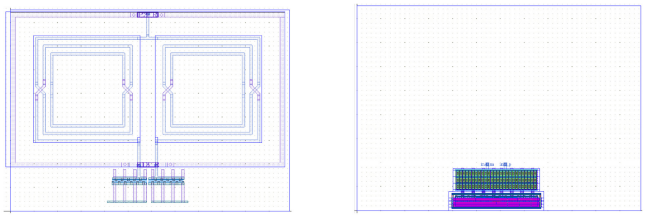


FIGURE 13. Output driver testbench schematic, highlighting the partial DUTs used for RC extraction and EM simulation.



(a) sub DUT for EM simulation

(b) sub DUT for parasitic extraction

FIGURE 14. Partially laid-out sub-blocks.

routing as a partially laid-out block. Fig. 14(a) shows the BAG-generated inductors' layout with routed leads (boxed in yellow in Fig. 13), used for EM simulations done with the EMX simulator. Separately, BAG3++ can generate a partial circuit for testing the transistors and resistors of the output driver (boxed in blue in Fig. 13). This circuit without magnetics is shown in Fig. 14(b).

B. CHARACTERIZATION SETUP AND DESIGN SCRIPT

A transient simulation is set up for the testbench in Fig. 13. A VerilogA random input bit-stream module with differential 0.8 V_{pp} is applied to the DUT inputs. The Spectre simulation results are then analyzed by using the BAG3++ *EyeAnalysis* class, which post-processes transient simulation results, creates eye diagrams, and reports the eye width and height.

The configuration settings of the *OptDesigner* class for this output driver are shown in Table 2. There is a distinction between design variables and simulation variables shown in the table. Design variables refer to layout parameters that cannot be changed after tape-out. These variables therefore must optimized across all process, voltage and temperature (PVT) corners. Simulation variables refer to bias points that can be set after tape-out using on-chip or off-chip calibration options like digital-to-analog converters (DACs). These can be separately optimized for each PVT corner.

For the purposes of this demonstration, the design variables are the number of fingers of the differential pair (seg_gm) and the tail transistor (seg_tail) for the DUT for extraction (blue box of Fig. 13), and the radius of the

TABLE 2. ***OptDesigner*** **input specifications. Resolution units refer to the technology-specific minimum layout drawn resolution.**

Parameters		Sweep / Target values
Corners		TT@25 °C, SS@−40 °C, FF@125 °C
Design variables	seg_gm	56, 64, 72, 80
	seg_tail	56, 64, 72, 80
	radius (resolution units)	18000, 22000, 26000, 30000
Simulation variables	V_{incm} (V)	0.65, 0.7, 0.75, 0.8
	V_{tail} (V)	0.65, 0.7, 0.75, 0.8
Speed		7.7 ps UI (130 Gbps)
Goal		Maximize eye height
Constraints	Eye width	$\geq$ 5.7 ps (0.75 UI)

inductor (DUT for EM simulation, yellow box of Fig. 13), while all other layout parameters are kept constant at specific values to prune the search space. The simulation variables are the bias points for the input common mode of the differential pair (V_{incm}) and the gate of the tail transistor (V_{tail}).

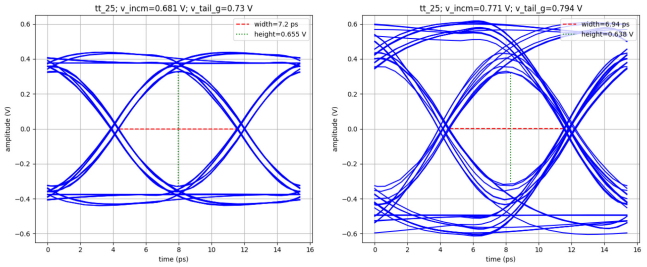
As there are 4 values being swept for each of the variables, this implies that there will be $4^3 = 64$ design points characterized at $4^2 = 16$ simulation settings across 3 PVT corners, resulting in a total of 3072 simulations to create the full characterization database. The characterization database stores results as *NumPy* arrays which can then be interpolated to predict the performance of other intermediate design and simulation points in the span of the search space.

C. RESULTS

To demonstrate process portability, *OptDesigner* was run using the specifications in Table 2 in two process technologies: a 16 nm FinFET process and a 45 nm SOI process. Designs converged to fixed design variables and corner-specific simulation variables, illustrated in Table 3. The optimizer guarantees the constraints across all corners and then optimizes for the worst given corner, i.e., FF 125 °C for the 16 nm FinFET process and SS −40 °C for the 45 nm SOI process. For each technology, the full 3072-point design space characterization has a 7-hour runtime using distributed parallel processing. The characterization database is then cached, so that subsequent optimization runs by querying the database are instantaneous.

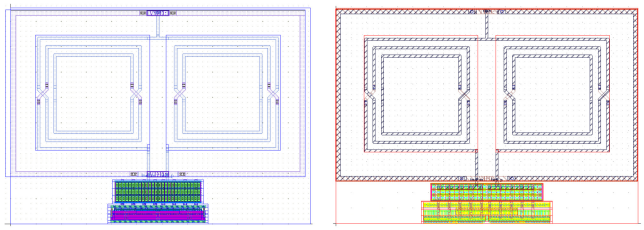
Fig 15 shows the corresponding eye diagrams for the TT 25 °C corner, using the bias conditions from Table 3. The optimal layouts are shown in Fig. 16. The design optimization converged to using the largest allowed sizes of the transistors to maximize the eye height in both technology nodes, since there were no constraints specified for the power consumption. While the achieved eye heights in both processes are similar, the SOI process exhibits a noisier eye, implying that the parasitics are larger and the performance is gain-bandwidth limited.

The cached database can be reused to explore optimal designs for additional constraints within the same design



(a) 16 nm FinFET (b) 45 nm SOI

FIGURE 15. **Eye diagrams for TT@ 25° corner based on Table 3.**



(a) 16 nm FinFET (b) 45 nm SOI

FIGURE 16. **Optimal layouts based on Table 3.**

TABLE 3. ***OptDesigner*** **results at V_{DD} = 1 V, maximizing eye height with no power constraints.**

Technology node		16 nm FinFET			45 nm SOI		
Corners		TT, 25	SS, -40	FF, 125	TT, 25	SS, -40	FF, 125
Results	Eye height (mV)	649	703	567	631	582	680
	Eye width (ps)	7.19	7.30	7.10	6.93	7.01	6.89
	Average current (mA)	18.3	19.5	18.5	21.1	17.0	26.5
Design variables	seg_gm	80			80		
	seg_tail	80			80		
	radius (res. units)	21768			18000		
Simulation variables	V_{incm} (mV)	681	744	681	771	771	800
	V_{tail} (mV)	730	800	730	794	794	800

space. Querying and optimizing for design points from the cached data takes less than 3 minutes. Table 4 shows *OptDesigner*'s results for the same 16 nm FinFET design space if a power constraint is added. The device sizing shrinks while the bias points are relatively unmoved, corresponding to maintain the same V^*. The inductor size also increases to achieve more swing. Table 5 shows results if the goal is to minimize power for a minimum eye height and width, within the same 16 nm FinFET design space. The devices shrink again, but the bias, particularly at the SS corner, is lower to achieve lower power for the eye height target.

V. OPEN-SOURCE EXTENSIBILITY WITH NGSPICE

In this section, we demonstrate the extensibility of BAG3++ to support additional custom tools that may be required by an end-user. Specifically described is the process through which support for Ngspice, an open-source simulator, was implemented in BAG3++ [15].

TABLE 4. *OptDesigner* results at V_{DD} = 1 V, maximizing eye height with power constraints in 16 nm FinFET.

Power Constraint		<= 16 mA			<= 14 mA		
Corners		TT, 25	SS, -40	FF, 125	TT, 25	SS, -40	FF, 125
Results	Eye height (mV)	482	519	440	440	471	404
	Eye width (ps)	7.48	7.45	7.51	7.50	7.47	7.52
	Average current (mA)	15.5	16.0	16.0	13.6	14.0	14.0
Design variables	seg_gm	80			64		
	seg_tail	64			56		
	radius (res. units)	26533			26045		
Simulation variables	V_{incm} (mV)	684	724	650	678	725	650
	V_{tail} (mV)	687	800	650	694	800	650

TABLE 5. *OptDesigner* results at V_{DD} = 1 V, minimizing power with minimum eye height in 16 nm FinFET.

Eye Height		>= 450 mV			>= 400 mV		
Corners		TT, 25	SS, -40	FF, 125	TT, 25	SS, -40	FF, 125
Results	Eye height (mV)	493	488	450	438	420	400
	Eye width (ps)	7.48	7.44	7.51	7.50	7.46	7.52
	Average current (mA)	16.5	14.2	16.8	13.7	11.5	13.9
Design variables	seg_gm	80			62		
	seg_tail	60			56		
	radius (res. units)	22491			25902		
Simulation variables	V_{incm} (mV)	684	684	650	684	684	650
	V_{tail} (mV)	774	774	726	750	746	752

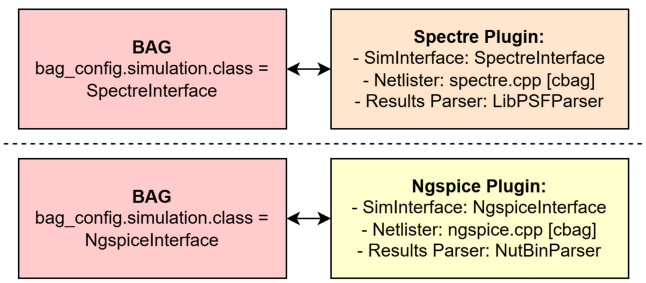


FIGURE 17. Configuring BAG to use different simulator plugins.

```yaml
meas_params:
  sim_envs: [tt_25]
  plot: true
  tbm_specs:
    sim_envs: [tt_25]
    sim_params:
      v_dc: 0.5
      v_bias: 0.7
    sim_options: {}
    sweep_var: 'VDC1'  # Ngspice
    # sweep_var: 'v_dc'  # Spectre
    sweep_options:
      type: LINEAR
      start: 0
      stop: v_VDD
      num: 51
    # ...
```

(a) Changes to OTA gain-bandwidth measurement parameters. Spectre supports sweeping parameters, whereas Ngspice only supports sweeping sources, so the variable `sweep_var` must be changed.

```python
def get_array_val_list(self) -> List[Mapping[str, Any]]:
    if self._use_spectre:
        return self.get_array_val_list_spectre()
    elif self._use_ngspice:
        return self.get_array_val_list_ngspice()
    else:
        raise NotImplementedError("Unknown simulator")

def get_array_val_list_spectre(self) -> List[Mapping[str, Any]]:
    ans = []
    for i, fname in enumerate(self.fname_list):
        td = self.soft_eval_expr(f'{self.td} + {i} * ({self.array_td_step})')
        ans.append(dict(fileName=f'\"{fname}\"', td=td))
    return ans

def get_array_val_list_ngspice(self) -> List[Mapping[str, Any]]:
    ans = []
    data = self._data
    for pwlf_data in data:
        param_str = ''
        for idx in range(pwlf_data.shape[0]):
            t, d = pwlf_data[idx, :]
            param_str += f" '{t}' '{d}'"
        ans.append(dict(value=param_str))
    return ans
```

(b) Changes to overdrive recovery test code. Spectre supports files for PWLFs, whereas Ngspice only supports time-value pairs, so new code must be added to produce this output.

FIGURE 18. Changes to generators for supporting Ngspice.

Implementing a BAG3++ plugin for any simulator requires developing three parts: a simulator-specific netlister, a simulator interface, and a results parser. These components ensure that a simulator can be written to and parsed from by the framework. The details of exact implementation will depend on the respective third-party simulator, though the illustrated steps are generic (Fig. 17). First, the netlister translates a circuit instance from BAG3++'s intermediate cellview representation into a simulator-specific netlist. For Ngspice specifically, a netlister for the SPICE simulator language is implemented. Second, a simulator interface must write simulation steps (e.g., setting up an AC frequency sweep) and also run simulations in accordance with a simulator's exposed API. Our simulator interface was extended to be compatible with Ngspice's analysis commands and routines. For example, the Ngspice interface uses the TRNOISE source for transient noise, which is defined and behaves differently from other simulators such as Spectre. Finally, the results parser translates the simulator output to raw data and then saved to a generic container (e.g., HDF5) for reuse later. The raw output of Ngspice is SPICE's Nutmeg binary format or *nutbin*.

The addition of Ngspice support to the core framework consists of about 2000 lines of code, of which less than 300 lines are completely original. This is because much of the structure of the Ngspice plugin mirrors BAG3++'s existing Spectre plugin counterpart and thus requires a relatively limited number of new lines of code.

To demonstrate the reuse of generator code between simulators, we tested using both Spectre and Ngspice with two circuit characterizations: an gain-bandwidth measurement for an OTA and an overdrive recovery test for a StrongARM latch, both in the Skywater 130 process. Changing from

Spectre to Ngspice requires no circuit generator modifications and only a few lines of simulator configuration modification (Fig. 18): the gain-bandwidth measurement required 1 line of code change (sweep parameter name), and the overdrive recovery test required 10 lines of code change (formating the piece-wise linear function (PWLF)). The results agree up to 5 significant figures, with further accuracy left to finer tuning of the simulator plugin or the simulation engines themselves. This shows that extending BAG for open-source tools is not only possible but can be done in a way that enables significant reuse of existing generators.

VI. CONCLUSION

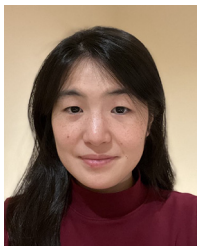
In this paper, we have presented BAG3++. BAG3++ creates a unified design environment that seamlessly transitions between schematic, layout, and verification views and affords flexibility and efficiency at all points in AMS design process. We have introduced faster, user-friendly, and open-sourced design compilation procedures and simulator interfaces that offer integration with EM simulators and various open-source tools. These new improvements permit automated design scripts and the introduction of the *OptDesigner* automated sizing feature. We have demonstrated the practical use of these new features through (1) a silicon-proven, bit-reconfigurable SAR-ADC implemented in the open-source Skywater 130nm process and (2) an ultra-high speed output driver optimized in Intel 16 nm and a 45 nm SOI process. Finally, we have demonstrated the extensibility of BAG3++ to open-source frameworks through the integration of the Ngspice simulator. BAG3++ documentation and open-source repositories can be accessed through [16].

ACKNOWLEDGMENT

The authors would like to thank Andrei Vladimirescu for his valuable feedback on this manuscript. They would also like to acknowledge the students and staff of the Berkeley Wireless Research Center for their contributions.

REFERENCES

[1] K. Kunal et al., "INVITED: ALIGN—Open-source analog layout automation from the ground up," in *Proc. 56th ACM/IEEE Design Autom. Conf. (DAC)*, 2019, pp. 1–4.
[2] T. Ajayi et al., "An open-source framework for autonomous SoC design with analog block generation," in *Proc. IFIP/IEEE 28th Int. Conf. Very Large Scale Integr. (VLSI-SOC)*, 2020, pp. 141–146.
[3] E. Chang et al., "BAG2: A process-portable framework for generator-based AMS circuit design," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, 2018, pp. 1–8.
[4] J. Crossley et al., "BAG: A designer-oriented integrated framework for the development of AMS circuit generators," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2013, pp. 74–81.
[5] C. Levy, "AMS generation frameworks: An industry perspective," presented at the IEEE Symp. VLSI Technol. Circuits, Honolulu, Hawaii, USA, 2024.
[6] M. Videnovic-Misic, "Programmatic analog IC design: MOSAIC bootcamp [society news]," *IEEE Solid-State Circuits Mag.*, vol. 14, no. 4, pp. 65–68, Nov. 2022.
[7] "libpsf." 2022. [Online]. Available: https://pypi.org/project/libpsf/
[8] "SciPy." 2024. [Online]. Available: https://docs.scipy.org/doc/scipy/index.html
[9] A. M. Abo and P. R. Gray, "A 1.5 V, 10-bit, 14 MS/s CMOS pipeline analog-to-digital converter," in *Symp. VLSI Circuits. Tech. Dig.*, 1998, pp. 166–169.
[10] T. Ogawa, H. Kobayashi, M. Hotta, Y. Takahashi, H. San, and N. Takai, "SAR ADC algorithm with redundancy," in *Proc. IEEE Asia Pac. Conf. Circuits Syst.*, 2008, pp. 268–271.
[11] M. Liu, X. Tang, K. Zhu, H. Chen, N. Sun, and D. Z. Pan, "OpenSAR: An open source automated end-to-end SAR ADC compiler," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Munich, Germany, 2021, pp. 1–9.
[12] L. D. R. Beal, D. R. Hill, R. A. Martin, and J. D. Hedengren, "GEKKO optimization suite," *Processes*, vol. 6, no. 8, p. 106, 2018.
[13] "Skywater130 SAR ADC." 2023. [Online]. Available: https://github.com/ucb-art/skywater130_bag3_sar_adc
[14] A. Biswas, "Design methodologies and automated generation of ultra high speed wireline SerDes transmitters," Ph.D. dissertation, Electr. Eng. Comput. Sci., Univ. California, Berkeley, CA, USA, 2023.
[15] "Ngspice." 2024. [Online]. Available: https://ngspice.sourceforge.io/
[16] "BAG3++ documentation." 2023. [Online]. Available: https://bag3-readthedocs.readthedocs.io/en/latest/



FELICIA GUO (Graduate Student Member, IEEE) received the B.S. degree in electrical and computer engineering from Cornell University, Ithaca, NY, USA in 2020. She is currently pursuing the Ph.D. degree in electrical engineering and computer sciences with the University of California at Berkeley, Berkeley, CA, USA.

She has previously held internships with Apple and Cadence Design Systems. Her research interests are in AMS design methodology and automation.



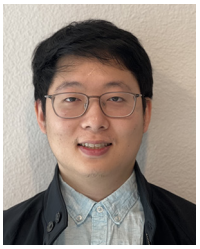
BOB ZHOU (Graduate Student Member, IEEE) received the B.S. degree in electrical engineering and computer science from the University of California at Berkeley, Berkeley, CA, USA, in 2018, where he is currently pursuing the Ph.D. degree with the Berkeley Wireless Research Center.

He has previously worked with Apple and Blue Cheetah Analog Design. His research interests include wireline link architecture and modeling, high speed AMS design, and design automation and optimization.



AYAN BISWAS (Member, IEEE) received the B.Tech. degree (Hons.) in electronics and electrical communication engineering from the Indian Institute of Technology, Kharagpur, India, in 2017, and the Ph.D. degree in electrical engineering and computer sciences from the University of California at Berkeley, Berkeley, CA, USA, in 2023.

He currently works as a Senior Engineer in analog and mixed signal design with Ayar Labs, San Jose, CA, USA. Previously he has interned with Blue Cheetah Analog Design. His research interests include design of high speed AMS circuit front-ends for optical communication systems, and automation of design optimization.



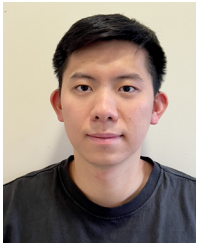
PAUL KWON received the B.S. and Ph.D. degrees in electrical engineering and computer sciences from the University of California at Berkeley, Berkeley, CA, USA, in 2017 and 2023, respectively.

He joined Blue Cheetah Analog Design, Sunnyvale, CA, USA, in 2023. He has previously interned with Apple. His research interests include die-to-die interconnects and the design automation of analog/mixed-signal circuits.



ZHAOKAI LIU (Member, IEEE) received the B.S. degree in microelectronics from the School of Electronics Engineering and Computer Science, Peking University, Beijing, China, in 2017, and the M.S. and Ph.D. degrees in electrical engineering and computer sciences from the University of California at Berkeley, Berkeley, CA, USA, in 2020 and 2023, respectively.

Since 2023, he has been with Intel Labs, Santa Clara, CA, USA. His research interests include analog design automation, high-speed analog-to-digital converters, silicon photonics, and high-speed electrical links.



KEN HO (Graduate Student Member, IEEE) received the B.S. degree in electrical and computer engineering from Cornell University, Ithaca, NY, USA in 2020. He is currently pursuing the Ph.D. degree in electrical engineering and computer sciences with the University of California at Berkeley, Berkeley, CA, USA.

He has previously held an internship with Intel Labs under Strategic CAD. His current research interests are in modeling and design methodologies for AMS circuits.



VLADIMIR STOJANOVIĆ (Fellow, IEEE) received the Dipl. Ing. degree from the University of Belgrade, Serbia, in 1998, and the Ph.D. degree in electrical engineering from Stanford University in 2005.

He is the CTO of Ayar Labs and an Adjunct Professor of Electrical Engineering and Computer Sciences with the University of California at Berkeley, Berkeley. He was also with Rambus, Inc., Los Altos, CA, USA, from 2001–2004, with MIT as an Associate Professor from 2005 to 2013, and with the University of California at Berkeley, Berkeley, as a Professor from 2013 to 2024. He is a Co-Founder of Ayar Labs, NanoSemi, and Numericcal. He received the 2006 IBM Faculty Partnership Award, and the 2009 NSF CAREER Award as well as the 2008 ICCAD William J. McCalla, the 2008 IEEE Transactions on Advanced Packaging, the 2010 ISSCC Jack Raper Best Paper and the 2021 ISSCC Best Forum Presenter Awards. He was an IEEE Solid-State Circuits Society Distinguished Lecturer from 2012 to 2013.



BORIVOJE NIKOLIĆ (Fellow, IEEE) received the Dipl.-Ing. and M.Sc. degrees in electrical engineering from the University of Belgrade, Belgrade, Serbia, in 1992 and 1994, respectively, and the Ph.D. degree from the University of California at Davis, Davis, CA, USA, in 1999.

In 1999, he joined the Department of Electrical Engineering and Computer Sciences, University of California at Berkeley, Berkeley, CA, USA, where he is currently a Professor and holds a National Semiconductor Distinguished Professorship in engineering.

Dr. Nikolić received the NSF CAREER Award in 2003, the College of Engineering Best Doctoral Dissertation Prize, the Anil K. Jain Prize for the Best Doctoral Dissertation in Electrical and Computer Engineering at the University of California at Davis in 1999, and the City of Belgrade Award for the Best Diploma Thesis in 1992. For work with his students and colleagues, he has received the Best Paper Awards at the IEEE International Solid-State Circuits Conference, the Symposium on VLSI Circuits, the IEEE International SOI Conference, the European Solid-State Device Research Conference, the European Solid-State Circuits Conference, the IEEE Custom Integrated Circuits Conference, the S3S Conference, the Design Automation Conference, and the ACM/IEEE International Symposium of Low-Power Electronics. From 2014 to 2015, he was a Distinguished Lecturer of the IEEE Solid-State Circuits Society. He was a Technical Program Chair for the 2022 Symposium on VLSI Technology and Circuits, and the General Chair for the 2024 IEEE Symposium on VLSI Technology and Circuits.